État de l'art des attaques contre les ML-NIDS

Synthèse du corpus `docs/attaques_concretes_ML_NIDS` (2021–2026)

1. Périmètre et classification

Un système de détection d'intrusions réseau fondé sur l'apprentissage automatique (*ML-NIDS*) transforme des paquets en caractéristiques : durées, tailles, nombres de paquets, indicateurs TCP ou relations entre machines. Un modèle (SVM, réseau de neurones, autoencodeur ou réseau sur graphes) utilise cette représentation pour reconnaître les activités malveillantes. Les attaques adversariales visent cette chaîne de détection : elles doivent être distinguées de l'activité réseau sous-jacente, par exemple un scan, un déni de service ou une force brute, qu'elles cherchent à dissimuler.

Trois familles se distinguent par leur moment d'action et leur objectif :

— **Évasion :** à l'inférence, modifier une communication malveillante pour qu'un détecteur déjà entraîné la considère comme bénigne.
— **Empoisonnement :** altérer les données ou leurs étiquettes avant un entraînement ou un réentraînement pour dégrader les décisions futures.
— **Porte dérobée :** forme ciblée d'empoisonnement qui apprend au modèle à produire une décision choisie lorsqu'un *déclencheur* apparaît, tout en conservant de bonnes performances sur les données sans déclencheur.

Cette classification se croise avec la connaissance de l'adversaire : **boîte blanche** (accès au modèle et à ses paramètres), **boîte grise** (informations partielles) ou **boîte noire** (sans accès interne). En boîte noire, l'adversaire peut exploiter des réponses du détecteur ou transférer des perturbations préparées sur un modèle substitut [1, 5, 6]. Il faut également préciser son contrôle : trafic émis, machines compromises, données d'apprentissage ou accès aux prédictions.

2. Évasion : des caractéristiques aux communications

Perturbation de caractéristiques. TIKI-TAKA compare cinq attaques en boîte noire (NES, Boundary, Pointwise, HopSkipJump et OPT) contre des MLP, CNN et C-LSTM [6]. Les requêtes guident la recherche d'un vecteur classé comme bénin ; une variante restreint les modifications aux variables temporelles. Mao et al. proposent SPTS, qui impose des contraintes et des relations entre caractéristiques, puis DGM, qui améliore le transfert entre modèles [5]. Leur démarche inclut un passage aux paquets. Toutefois, un vecteur adversarial valide mathématiquement ne garantit pas qu'une communication correspondante soit réalisable et conserve son effet malveillant.

Remodelage du trafic. TrafficManipulator agit sur les délais et ajoute des paquets pour modifier les statistiques observées, notamment contre KitNET, un ensemble d'autoencodeurs [1]. TANTRA apprend les intervalles entre paquets du trafic bénin avec un LSTM, puis adapte la temporisation du trafic malveillant sans en changer le contenu [2]. Ce LSTM est le générateur de l'attaque ; les détecteurs ciblés comprennent KitNET, un autoencodeur et Isolation Forest. Le coût en latence reste une contrainte, en particulier pour les activités dépendantes du débit.

Transformation guidée par l'explicabilité. Okada et al. utilisent KernelSHAP pour identifier les caractéristiques influentes d'un MLP en boîte noire, puis les modifier par des actions sur le trafic [3]. Selon l'extracteur, une force brute FTP est adaptée en modifiant les indicateurs PSH ou en séparant les tentatives dans des sessions TCP distinctes. L'étude évalue aussi des attaques XSS entre machines virtuelles et vérifie la préservation de l'activité malveillante : elle relie explicitement évasion du modèle et fonctionnement de l'attaque réseau.

Perturbation structurelle. Pour E-GraphSAGE et LineGraphSAGE, Venturi et al. modifient les communications représentées par les arêtes et les nœuds du graphe [4]. La variante C2x_B ajoute des communications bénignes depuis un hôte compromis, sans retirer le flux malveillant : le voisinage observé change et peut tromper le modèle. Ces expériences sur flux et graphes ne constituent cependant pas une validation complète en réseau actif.

3. Empoisonnement et portes dérobées

Dégradation de l'apprentissage. Venkatesan et al. étudient un SVM distinguant trafic bénin et scans Nmap [7]. L'adversaire insère des exemples étiquetés « scan » près d'exemples bénins reconnus avec une forte confiance. Lors du réentraînement, ces contradictions déplacent la frontière de décision et augmentent les erreurs, notamment les faux positifs. L'attaque vise ici la disponibilité du détecteur, plutôt que seulement l'évasion d'un flux particulier. L'étude impose des contraintes de cohérence aux caractéristiques, mais ne démontre pas une chaîne complète de rejeu réseau. Une condition essentielle est donc l'accès à une partie des données d'apprentissage : envoyer du trafic à un modèle figé ne suffit pas à l'empoisonner.

Comportement conditionné par un déclencheur. PCAP-Backdoor insère des paquets construits dans une partie du trafic bénin utilisé pour l'entraînement, en conservant les étiquettes initiales : c'est un empoisonnement à *étiquettes propres* [8]. Le modèle apprend une association entre les statistiques induites par ce déclencheur et la classe bénigne. À l'inférence, l'ajout du déclencheur au trafic malveillant provoque alors une classification bénigne. À la différence de l'empoisonnement précédent, les résultats sur trafic sans déclencheur peuvent rester bons : une évaluation standard ne suffit donc pas à révéler la compromission. L'étude agit sur les PCAP et examine plusieurs modèles et jeux de données IoT et industriels.

4. Enseignements, défenses et critères d'évaluation

Le corpus montre que la vulnérabilité dépend du **couple modèle–extracteur** et des capacités de l'adversaire. Les GNN introduisent une surface d'attaque structurelle ; les détecteurs fondés sur des statistiques temporelles peuvent être sensibles à des changements de rythme. Les résultats obtenus sur Kitsune, CICIDS, TON_IoT ou CTU-13 restent liés au protocole expérimental : des taux de succès issus d'études différentes ne sont pas directement comparables.

Les défenses étudiées comprennent l'entraînement adversarial et les ensembles de modèles, la détection des requêtes répétées [1, 6], ainsi que la sanitisation des données par désaccord entre classifieurs [7]. PCAP-Backdoor souligne également les limites du regroupement des activations pour identifier certains déclencheurs [8]. Ces méthodes atténuent des scénarios précis, sans établir une robustesse universelle.

Pour le démonstrateur, une expérience doit comparer un état de référence et un état attaqué, avec le **rappel du trafic malveillant**, les **faux positifs** et le **coût de l'attaque** : délai, volume, nombre de requêtes ou fraction de données empoisonnées. Le taux d'évasion doit préciser son dénominateur, notamment si seuls les flux initialement détectés sont retenus. Pour une porte dérobée, il faut mesurer séparément le succès avec déclencheur et la qualité sans déclencheur. Enfin, les caractéristiques doivent être réextraites après transformation des paquets et la réussite de l'activité réseau doit être vérifiée. TrafficManipulator/KitNET, TANTRA et les scénarios d'Okada constituent ainsi des pistes complémentaires pour illustrer l'évasion ; le SVM empoisonné et PCAP-Backdoor couvrent les risques liés à l'apprentissage.

Références du corpus

[1] D. Han et al. (2021). *Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors*. IEEE JSAC. doi:10.1109/JSAC.2021.3087242.

[2] Y. Sharon et al. (2022). *TANTRA : Timing-Based Adversarial Network Traffic Reshaping Attack*. IEEE TIFS. doi:10.1109/TIFS.2022.3201377.

[3] S. Okada et al. (2025). *Xai-driven black-box adversarial attacks on network intrusion detectors*. IJIS. doi:10.1007/s10207-025-01016-0.

[4] A. Venturi, D. Stabili et M. Marchetti (2024). *Problem space structural adversarial attacks for Network Intrusion Detection Systems based on Graph Neural Networks*. arXiv:2403.11830v2.

[5] Z. Mao et al. (2025). *Exploring transferable adversarial attacks for Deep Learning-based Network Intrusion Detection*. JNCA. doi:10.1016/j.jnca.2025.104255.

[6] C. Zhang, X. Costa-Pérez et P. Patras (2022). *Adversarial Attacks Against Deep Learning-Based Network Intrusion Detection Systems and Defense Mechanisms*. IEEE/ACM ToN. doi:10.1109/TNET.2021.3137084.

[7] S. Venkatesan et al. (2021). *Poisoning Attacks and Data Sanitization Mitigations for Machine Learning Models in Network Intrusion Detection Systems*. MILCOM. doi:10.1109/MILCOM52596.2021.9652916.

[8] A. Koyatan Chathoth et al. (2026). *PCAP-Backdoor : Backdoor Generator in Network Traffic for Intrusion Detection Systems*. ACM TCPS. doi:10.1145/3805033.